

THEMIS Development Reference

0. Introduction

Current Version

- Manual version: v0.2.0
- Compatible Minimum AOS version: v0.2.0
- Compatible Minimum AOS Server version: v0.2.0

1. System and Dependencies

- **Main Computer**

The main computer comes with Ubuntu 22.04.5 LTS (Jammy Jellyfish) preinstalled. Python 3.10.12 and all required dependencies are preconfigured e.g., pip, NumPy, pyserial, termcolor, numba, posix ipc, lcm. ROS2 Humble is also installed.

- **Head Vision Computer**

The following core software packages are preinstalled and configured on the head vision computer:

- [JetPack 6.2](#)
- ROS2 Humble
- [ZED SDK for NVIDIA Jetson](#)

- **Front Vision Computer**

The Front Vision Computer has a similar configuration to the Head Vision Computer. The main difference is that the body vision computer also includes the ZED X driver required for the ZED X Mini camera. Key packages for the Front Vision Computer are:

- [JetPack 6.2](#)
- ROS2 Humble
- [ZED SDK for NVIDIA Jetson](#)
- ZED Link Driver

2. AOS Software Introduction

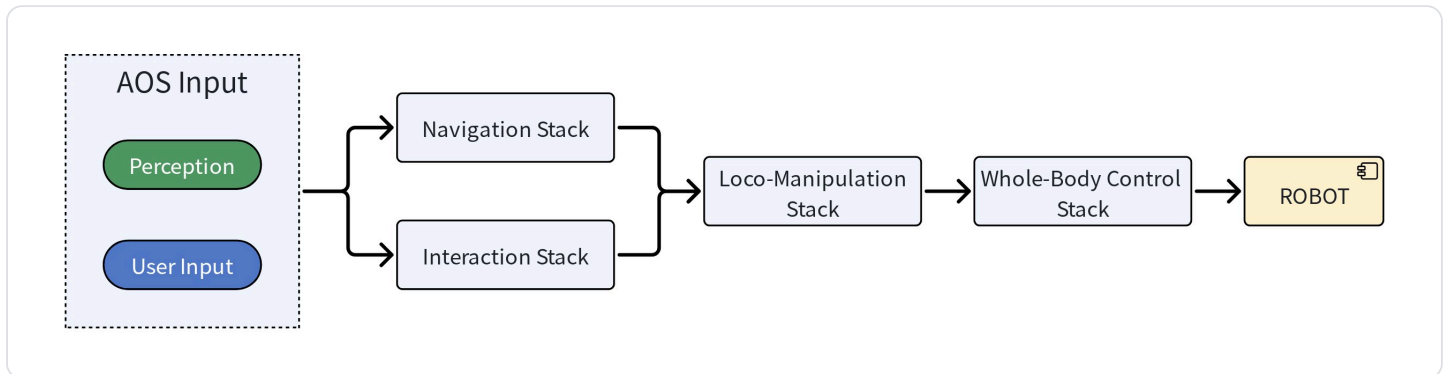


SOFTWARE SUPPORT

Every THEMIS unit is shipped with the latest software release installed and fully tested. The software is continuously maintained and updated at <https://github.com/Westwood-Robotics/AOS>.

2.1 AOS Framework

The AI-Augmented Humanoid Operating System (AOS) is ...



AOS Framework

2.2 AOS Inputs

AOS requires both self-motion awareness and environmental perception. AOS uses multiple sensing technologies, which can be categorized into two types. Proprioceptive sensors measure values internal to the robot system, e.g., inertial measurement unit (IMU) and joint encoders. Exteroceptive sensors acquire information from the robot's environment, e.g., cameras. By combining these sources of information, AOS can then accept user commands and execute specific tasks, e.g., moving to a target location or detecting a target object.

2.3 Navigation Stack

The navigation stack works as follows: visual odometry provided by the ZED camera system provides a global transform tree. Based on this, the camera sensors generate a robot-centric local map. The system can navigate with or without a global map: the user specifies a goal point, and the navigation stack outputs either velocity commands or a planned path to the loco-manipulation stack for execution.

2.4 Interaction Stack

Interaction Module Overview

Detection Boxes:

The interaction module can recognize object classes on which the model has been trained and mark them with detection boxes in the image. Currently, it can identify categories such as green bottles, green cylindrical objects, Nongfu Spring mineral water bottles, and blue cola cans. At 720p, it can achieve accurate recognition within a distance of 1.5 m.

Depth-Based Size Classifier:

Within a range of 1.5 m, it can accurately classify objects into two size categories: 500 mL and 1000 mL–3000 mL. The detected object’s size is also labeled on its detection box.

Target Point:

By clicking any position in the image, a target point can be generated for navigation. The robot will then navigate to this target position, published on the `/target_pose` topic.

If the click is on an object within a detection box, the target point will track the object’s movement within a 50 cm range, ensuring that the target point stays aligned with the object. However, due to camera localization inaccuracies, tracking may become unreliable or fail under certain conditions.

2.5 Loco-Manipulation Stack

Using the continuously updated path from the navigation stack together with commands from the interaction stack, the loco-manipulation stack generates short-horizon whole-body reference trajectories which respect the kinematics and dynamics of the robot.

2.6 Whole-Body Control Stack

The goal of the whole-body control (WBC) stack is to compute, in real time, the joint torques, velocities, and positions required to satisfy the robot's kinematic and dynamic constraints. These commands are then sent to the robot's joint drivers for execution.

3. Developing with THEMIS

3.1 User Login

To access the onboard computers, users can first logon with external monitors, mice, and keyboards. The default logon information is listed in Table 3.1. After configuring Wi-Fi, users can also access the onboard computers remotely via SSH (Secure Shell).

Table 3.1: Onboard Computer Logon Information

	Host Name	IP Address	Logon ID	Logon Password
Main Computer	themisxxx	192.168.0.xx1	themis	themis

Head Jetson	headxxx	192.168.0.xx2	nvidia	nvidia
Front Jetson	frontxxx	192.168.0.xx3	nvidia	nvidia

3.2 API and ROS Topics

This section explains how AOS supports both ROS interfaces and non-ROS APIs, and how the two are related, helping users choose the most appropriate development approach. Our system provides two interfaces for users:

ROS 2 Topic Interface – All system data are published as ROS 2 topics. Users familiar with ROS 2 can directly subscribe to these topics to access the required information.

Non-ROS API Interface – Built on top of the ROS 2 topic interface, AOS also provides a ROS-independent wrapper API that is independent of ROS. This interface exposes only basic data types, allowing users with no ROS background to easily access system data.

3.3 AOS Input API

3.3.1 Perception API

The perception module provides access to raw and processed sensor data from two onboard cameras. Both cameras publish their perception data via ROS topics, enabling users to subscribe directly and retrieve information such as point clouds and image streams.

The two perception sources are:

- **Front Camera:** ZED X Mini (mounted on the front of the body)
- **Head Camera:** ZED 2i (mounted on the head)

Each camera operates as an independent ROS node and publishes its own perception data, including but not limited to:

- **RGB images:** `zed/zed_node/rgb/image_rect_color`
- **Depth images:** `zed/zed_node/depth/depth_registered`
- **Point clouds:** `zed/zed_node/point_cloud/cloud_registered`

Users can subscribe to these ROS2 topics to integrate perception data into their applications. The complete list of available topics, message formats, and API usage can be found in the [ZED ROS2 documentation](#) provided by Stereolabs.

3.3.2 User Input API

TBC

3.4 Navigation API

The navigation module is built on the AOS navigation stack. The API provides interfaces for setting navigation goals, adjusting tolerance thresholds, tuning planner and controller parameters, and receiving navigation feedback. Developers can use these APIs to build custom applications and tune system parameters.

Detailed API documentation is available in [📖 AOS & THEMIS API](#)

3.4.1 Maps

- **CostMap**

Determines traversability geometrically, classifying obstacles by height.

- **Global Map**

The global map is created via SLAM. Before starting a new task, users can build a map of the environment from the **Dynamic Mapping** panel in the UI; refer to the [📖 AOS Controller Manual](#) for details. Note that the map origin is defined at the position where the robot starts mapping.

- **Implicit Semantic Map**

A sub-layer of the cost map, used to identify obstacles that geometric analysis alone cannot distinguish,

so that the robot can travel safely.

3.4.2 Sending Navigation Goals

To initiate a navigation task, users can publish a target pose message to the topic `/target_pose`. This message typically contains both position and orientation expressed in the selected coordinate frame, typically the `map` frame. Once received, the navigation system interprets this pose as the active goal for the robot to reach.

By default, the current target pose remains valid until one of the following conditions is met:

Goal Reached – The robot arrives within the configured tolerance thresholds (position and orientation), in which case the navigation system reports success.

New Target Pose Published – If a subsequent message is published to `/target_pose`, the system compares it against the active goal.

- If the new pose differs from the current goal by less than a specified threshold in position and orientation, the navigation system treats it as a duplicate and continues toward the existing goal.
- If the difference exceeds the threshold, the new pose is accepted as the updated navigation goal, and the planner immediately reconfigures the path toward this new destination.

3.4.3 Dynamically adjust planner and controller parameters

Parameters can typically be updated through:

ROS 2 Services or Dynamic Reconfigure Interfaces – Users can call parameter-update services (e.g., `/planner_server/set_parameters`, `/controller_server/set_parameters`) with new values.

API Calls or Config Files – External systems may use the Navigation API bindings (C++, Python) to push updated parameter sets.

Once updated, the navigation nodes (planner and controller) immediately reconfigure themselves with the new settings, without interrupting the ongoing navigation task.

3.4.4 Subscribe to navigation status and feedback

Topic: `/navigate_to_pose/_action/status`

Message Type: `action_msgs.msg/GoalStatusArray`

According to GoalStatus message documentation, users can get the relationship between codes and states. For example, status code "2" means "executing", "4" means "succeeded", and "6" means "failed".

Get Navigation Status

```
1 self.subscription = self.create_subscription(  
2     GoalStatusArray,  
3     '/navigate_to_pose/_action/status',  
4     self.status_callback,  
5     10)
```

3.5 Interaction API

3.5.1 Object Detection API

topic	Function description	API	data type
/detected_objects	Publish detected object information, including object ID, category, confidence, and bounding box	aos_api->getDetectedObjects()	<code>int id;</code> <code>std::string category;</code> <code>float score;</code> <code>std::vector<float> bbox;</code>
/target_pose			<code>float x;</code>

	Publishes the target pose in the <code>map</code> frame, including 3D position and quaternion orientation.	<code>aos_api->getTargetPose()</code>	<pre>float y; float z; float qx; float qy; float qz; float qw;</pre>
<code>/grasp_points</code>	Publish 3D grasp point positions and poses for robotic gripping tasks	<code>aos_api->getGraspPoints()</code>	<pre>std::vector<float> position; std::vector<float> orientation;</pre>
<code>/principal_axes</code>	Publishes the object's 3D principal-axis information, including the centroid and directional axes, for object pose analysis	<code>aos_api->getPrincipalAxes()</code>	<pre>std::vector<float> centroid; std::vector<float> axis_x; std::vector<float> axis_y; std::vector<float> axis_z; std::vector<float> variances; std::string object_category; int object_id;</pre>
<code>/principal_axis_2d</code>	Publish 2D principal-axis information, including centroid and axis endpoints, for 2D image analysis	<code>aos_api->getMarkerPoint()</code>	<pre>std::vector<float> centroid; std::vector<float> axis_end;</pre>
<code>/grasp_points_2d</code>	Publishes 2D grasp point coordinates for grasp point positioning within the image plane	<code>aos_api->getPrincipalAxis2D()</code>	<pre>std::vector<std::vector<float>> points; // [[u, v], ...]</pre>
<code>/marker_point</code>	Publish the 2D coordinates and tracking status of the marker point in the image for visual tracking	<code>aos_api->getGraspPoints2D()</code>	<pre>float u; float v;</pre>

/yolo_loaded /navigation_status /grasp_status	Publishes the YOLO model loading status, navigation mode switching, and grabbing mode switching status	aos_api->getObjectDetectionStatus()	<pre>bool yolo_loaded; bool navigation_status; bool grasp_status;</pre>
/arrow_direction	Publish arrow direction vectors to indicate a specific direction or target when the target point cannot be projected	aos_api->getArrowDirection()	<pre>float x; float y; float z;</pre>

The detailed API code is as follows:

Detailed Detection API Definitions:

/detected_objects



```
aos_api->getDetectedObjects()
```

```
struct DetectedObjectData {
    struct Object {
        int id;
        std::string category;
        float score;
        std::vector<float> bbox; // [x_min, y_min, x_max, y_max]
    };
    std::vector<Object> objects;
    uint64_t timestamp_ms;
    uint64_t detection_timestamp_ms;
};
```

/target_pose



```
aos_api->getTargetPose()
```

```

struct TargetPoseData {
    float x;
    float y;
    float z;
    float qx;
    float qy;
    float qz;
    float qw;
    uint64_t timestamp_ms;
};

```

/grasp_points



```
aos_api->getGraspPoints()
```

```

struct GraspPointsData {
    struct Pose {
        std::vector<float> position; // [x, y, z]
        std::vector<float> orientation; // [x, y, z, w]
    };
    std::vector<Pose> poses;
    uint64_t timestamp_ms;
};

```

/principal_axes



```
aos_api->getPrincipalAxes()
```

```

struct PrincipalAxesData {
    struct Axes {
        std::vector<float> centroid; // [x, y, z]
        std::vector<float> axis_x; // [x, y, z]
        std::vector<float> axis_y; // [x, y, z]
        std::vector<float> axis_z; // [x, y, z]
    };
};

```

```

        std::vector<float> variances; // [var_x, var_y,
var_z]

        std::string object_category;

        int object_id;

    };

    Axes axes;

    uint64_t timestamp_ms;

};

```

/principal_axis_2d

 aos_api->getPrincipalAxis2D()

```

struct PrincipalAxis2DData {

    std::vector<float> centroid; // [x, y]

    std::vector<float> axis_end; // [x, y]

    uint64_t timestamp_ms;

};

```

/grasp_points_2d

 aos_api->getGraspPoints2D()

```

struct GraspPoints2DData {

    std::vector<std::vector<float>> points; // [[u, v],
... ]

    uint64_t timestamp_ms;

};

```

/marker_point

 aos_api->getMarkerPoint()

```

struct MarkerPointData {

    float u;

```

```
float v;  
bool is_tracked;  
uint64_t timestamp_ms;  
};
```

/yolo_loaded, /navigation_status/grasp_status



```
aos_api->getObjectDetectionStatus()  
struct ObjectDetectionStatusData {  
    bool yolo_loaded;  
    bool navigation_status;  
    bool grasp_status;  
    uint64_t timestamp_ms;  
};
```

/arrow_direction



```
aos_api->getArrowDirection()  
struct ArrowDirectionData {  
    float x;  
    float y;  
    float z;  
    uint64_t timestamp_ms;  
};
```

3.5.2 OC-VAM API

TBC

3.6 Loco-Manipulation API

Loco-manipulation software is not fully open-source, but APIs are provided to users for further development.

3.6.1 Locomotion API

The state estimator, low-level controller, and high-level controller are provided as precompiled binaries. To operate THEMIS, i.e., to change locomotion mode (standing or walking), walking velocity, base orientation, and CoM (center of mass) position, users can implement or customize the top-level command thread, e.g.: `Main`

`PC/THEMIS/THEMIS/Play/Locomotion/top_level.py`.

- Initialization

Terminal

```
1 from Play.Others import locomotion as lm_api
```

- Locomotion Mode

Locomotion Mode

```
1 """
2 Set locomotion mode
3
4 ARGUMENT:
5 m (int) - standing (0), walking (1)
6 """
7 lm_api.set_locomotion_mode(m)
```

- Walking Velocity

Walking Velocity

```
1 """
2 Set walking velocity
3
4 ARGUMENT:
5 v (1 x 2) - horizontal velocity
6 w (float) - yaw rate
7 """
8 lm_api.set_walking_velocity(v, w)
```

- Base Orientation

```

1  """
2  Set base orientation
3
4  ARGUMENT:
5  roll (float) - roll angle [rad]
6  pitch (float) - pitch angle [rad]
7  yaw (float) - yaw angle [rad]
8  """
9  lm_api.set_base_orientation(roll, pitch, yaw)

```

- CoM Position

CoM Position

```

1  """
2  Set com position
3
4  ARGUMENT:
5  x (float) - x position [m]
6  y (float) - y position [m]
7  z (float) - z position [m]
8  """
9  lm_api.set_com_position(x, y, z)

```



Command Safety Precautions

- Make sure the `top_level.py` is not running.
- Always test your code in simulation first.
- Make sure the command reference trajectory is smooth enough to avoid jittering.
- Make sure base orientation and CoM position are set back to zero before walking.
- Set the walking velocity to zero before switching to standing mode.
- Adjusting the base orientation or CoM position while the robot is walking is not recommended.

3.6.2 Manipulation API

TBC

3.7 Whole-Body Control API

3.7.1 Robot Model API

The complete robot model, including kinematic and dynamic computations, is implemented in the following file, which also contains detailed usage documentation: `Main PC/THEMIS/THEMIS/Library/ROBOT_MODEL/model.py`.

- Initialization

Initialization

```
1 from Library.ROBOT_MODEL import model as rm_api
```

- Robot Inverse Dynamics

```
ID
1  """
2  Compute joint-space equations of motion
3
4  ARGUMENT:
5  R_wb (3 x 3) - base rotation matrix in world frame
6  p_wb (1 x 3) - base position          in world frame
7  w_bb (1 x 3) - base angular rate     in base frame
8  v_bb (1 x 3) - base linear velocity in base frame
9  q_leg_r (1 x 6), q_leg_l (1 x 6), q_arm_r (1 x 7), q_arm_l (1 x 7), q_head
   (1 x 2) - joint positions
10 dq_leg_r (1 x 6), dq_leg_l (1 x 6), dq_arm_r (1 x 7), dq_arm_l (1 x 7),
   dq_head (1 x 2) - joint velocities
11
12 RETURN:
13 H      (34 x 34) - inertia matrix
14 CG     ( 1 x 34) - vector of centrifugal, Coriolis, and gravity terms
15 AG     ( 6 x 34) - centroidal momentum matrix
16 dAGdq ( 1 x  6) - time derivative of AG times dq
17 p_wg   ( 1 x  3) - CoM position in world frame
18 v_wg   ( 1 x  3) - CoM velocity in world frame
19 h      ( 1 x  6) - centroidal momentum, i.e., angular momentum about CoM and
   linear momentum
20 """
21 H, CG, AG, dAGdq, p_wg, v_wg, h = rm_api.get_robot_dynamics(R_wb, p_wb, w_bb,
   v_bb,
22                                     q_leg_r, q_leg_l,
   q_arm_r, q_arm_l, q_head,
23                                     dq_leg_r,
   dq_leg_l, dq_arm_r, dq_arm_l, dq_head)
```

- Leg Transformation Matrices

```

leg TF'''
2   Compute transformation matrices for each frame on leg in world frame
3
4   ARGUMENT:
5   leg      ( int) - leg index, i.e., right leg (+1), left leg (-1)
6   location ( int) - end frame location index, i.e., ankle (0), foot middle (1),
   inner toe (2), outer toe (3), heel (4)
7   T_wb     (4 x 4) - transformation matrix of base in world frame
8   q        (1 x 6) - leg joint positions
9
10  RETURN:
11  T (6 x 4 x 4) - transformation matrices of each frame on leg in world frame
12  '''
13  T = rm_api.get_leg_transformations(leg, location, T_wb, q)

```

- Arm Transformation Matrices

```

Arm TF
1   '''
2   Compute transformation matrices for each frame on arm in world frame
3
4   ARGUMENT:
5   arm      ( int) - arm index, i.e., right arm (+1), left arm (-1)
6   location ( int) - end frame location index, i.e., wrist (0), hand (1)
7   T_wb     (4 x 4) - transformation matrix of base in world frame
8   q        (1 x 7) - arm joint positions
9
10  RETURN:
11  T (7 x 4 x 4) - transformation matrices of each frame on arm in world frame
12  '''
13  T = rm_api.get_arm_transformations(arm, location, T_wb, q)

```

- Head Transformation Matrices

```

Head TF
1   '''
2   Compute transformation matrices for each frame on head in world frame
3
4   ARGUMENT:
5   head     ( int) - head index (0)
6   location ( int) - end frame location index, i.e., neck (0), camera (1)
7   T_wb     (4 x 4) - transformation matrix of base in world frame
8   q        (1 x 2) - head joint positions
9

```

```

10 RETURN:
11 T (2 x 4 x 4) - transformation matrices of each frame on head in world frame
12 """
13 T = rm_api.get_head_transformations(head, location, T_wb, q)

```

- Leg Inverse Kinematics

```

Leg IK
1  """
2  Compute leg joint positions given foot transformation matrix in base frame
   (check "solve_foot_IK" function in robot_data.py for how to use)
3
4  ARGUMENT:
5  leg      ( int) - leg index, i.e., right leg (+1), left leg (-1)
6  location ( int) - end frame location index, i.e., ankle (0), foot middle (1),
   inner toe (2), outer toe (3), heel (4)
7  T        (4 x 4) - transformation matrix of foot frame in base frame
8  q0       (1 x 6) - initial guess
9
10 RETURN:
11 q (4 x 6) - all possible solutions with a maximum number of 4
12 """
13 q = rm_api.get_leg_pose(leg, location, T, q0)

```

- Arm Inverse Kinematics

```

Arm IK
1  """
2  Compute arm joint positions given hand transformation matrix in base frame
   (check "solve_hand_IK" function in robot_data.py for how to use)
3
4  ARGUMENT:
5  arm      ( int) - arm index, i.e., right arm (+1), left arm (-1)
6  location ( int) - end frame location index, i.e., wrist (0), hand (1)
7  T        (4 x 4) - transformation matrix of foot frame in base frame
8  q0       (1 x 7) - initial guess
9
10 RETURN:
11 q (1 x 7) - all possible solutions with a maximum number of 1
12 """
13 q = rm_api.get_arm_pose(arm, location, T, q0)

```

- Foot Forward Kinematics

Foot FK

```
1  """
2  Compute transformation matrix and Jacobian for foot frame in base frame
3
4  ARGUMENT:
5  leg      ( int) - leg index, i.e., right leg (+1), left leg (-1)
6  location ( int) - end frame location index, i.e., ankle (0), foot middle (1),
   inner toe (2), outer toe (3), heel (4)
7  q        (1 x 6) - leg joint positions
8  dq       (1 x 6) - leg joint velocities
9
10 RETURN:
11 T        (4 x 4) - transformation matrix of foot frame in base frame
12 v        (1 x 6) - angular and linear velocity of foot frame in base frame
13 J        (6 x 6) - angular and linear Jacobian of foot frame in base frame
14 dJdq     (1 x 6) - time derivative of J times dq
15 """
16 T, v, J, dJdq = rm_api.get_foot_FK(leg, location, q, dq)
```

- Hand Forward Kinematics

Hand FK

```
1  """
2  Compute transformation matrix and Jacobian for hand frame in base frame
3
4  ARGUMENT:
5  arm      ( int) - leg index, i.e., right arm (+1), left arm (-1)
6  location ( int) - end frame location index, i.e., wrist (0), hand (1)
7  q        (1 x 7) - arm joint positions
8  dq       (1 x 7) - arm joint velocities
9
10 RETURN:
11 T        (4 x 4) - transformation matrix of hand frame in base frame
12 v        (1 x 6) - angular and linear velocity of hand frame in base frame
13 J        (6 x 7) - angular and linear Jacobian of hand frame in base frame
14 dJdq     (1 x 6) - time derivative of J times dq
15 """
16 T, v, J, dJdq = rm_api.get_hand_FK(arm, location, q, dq)
```

- Joint States

motor2joint

```
1  """
```

```

2  Convert motor states to joint states
3
4  ARGUMENT:
5  sim      ( int) - hardware or simulation, i.e., hardware (0), simulation (1)
6  chain    ( int) - limb chain index, i.e., head (0), right leg (+1), left leg
              (-1), right arm (+2), left arm (-2), right hand (+3), left hand (-3)
7  option   ( int) - state property, i.e., position (0), velocity (1), torque (2)
8  m        (1 x n) - motor states
9
10 RETURN:
11 q         (1 x n) - joint states
12 """
13 q = rm_api.get_joint_states(sim, chain, option, m)

```

- Motor States

```

joint2motor
1  """
2  Convert joint states to motor states
3
4  ARGUMENT:
5  sim      ( int) - hardware or simulation, i.e., hardware (0), simulation (1)
6  chain    ( int) - limb chain index, i.e., head (0), right leg (+1), left leg
              (-1), right arm (+2), left arm (-2), right hand (+3), left hand (-3)
7  option   ( int) - state property, i.e., position (0), velocity (1), torque (2)
8  q         (1 x n) - joint states
9
10 RETURN:
11 m         (1 x n) - motor states
12 """
13 m = rm_api.get_motor_states(sim, chain, option, q)

```

Furthermore, a detailed mathematical modeling of THEMIS is summarized in [Appendix A](#), including forward

kinematics, inverse kinematics, and dynamic properties.

3.7.2 Whole-Body Control API

- Initialization

Terminal

```

1  from Play.Others import wbc as wbc_api

```

- Get IMU States

Get IMU States

```
1  """
2  Get imu states
3
4  RETURN:
5  a (1 x 3) - linear acceleration
6  w (1 x 3) - angular rate
7  R (3 x 3) - rotation matrix
8  """
9  a, w, R = wbc_api.get_imu_states()
```

- Get Joint States

Get Joint States

```
1  """
2  Get joint states
3
4  ARGUMENT:
5  chain ( int) - limb chain index, i.e., head (0), right leg (+1), left leg
   (-1), right arm (+2), left arm (-2), right hand (+3), left hand (-3)
6
7  RETURN:
8  q (1 x n) - joint positions
9  dq (1 x n) - joint velocities
10 u (1 x n) - joint torques
11 t (1 x n) - motor temperatures
12 v (1 x n) - motor voltages
13 """
14 q, dq, u, t, v = wbc_api.get_joint_states(chain)
```

- Set Joint States

Set Joint States

```
1  """
2  Set joint states
3
4  ARGUMENT:
5  chain ( int) - limb chain index, i.e., head (0), right leg (+1), left leg
   (-1), right arm (+2), left arm (-2), right hand (+3), left hand (-3)
6  u (1 x n) - joint torques
7  q (1 x n) - joint positions
```

```

8 dq (1 x n) - joint velocities
9 kp (1 x n) - motor p gains
10 kd (1 x n) - motor d gains
11 """
12 wbc_api.set_joint_states(chain, u, q, dq, kp, kd)

```

- Disable Motors

Disable Motors

```

1 """
2 Disable motors
3
4 ARGUMENT:
5 chain (int) - limb chain index, i.e., head (0), right leg (+1), left leg (-1),
  right arm (+2), left arm (-2), right hand (+3), left hand (-3)
6 """
7 wbc_api.disable_motors(chain)

```

- Damping Motors

Damping Motors

```

1 """
2 Damping motors
3
4 ARGUMENT:
5 chain (int) - limb chain index, i.e., head (0), right leg (+1), left leg (-1),
  right arm (+2), left arm (-2)
6 """
7 wbc_api.damping_motors(chain)

```



BE CAREFUL WHAT YOU COMMAND

- Always test your code in simulation first.
- Make sure the command reference trajectory is smooth enough to avoid jittering.

3.8 Simulation

In the simulation environment, users operate the robot through the same APIs.

The simulation environment mimics the functionality of the hardware interface so that the AOS software architecture does not need to be adjusted for different environments. For example,

current sensor states can be retrieved from the simulation environment, and desired joint commands can be sent to the simulation environment in the same way as when using the physical robot. Meanwhile, to reduce the computational load, a simplified CAD model is used instead of the detailed one.

Currently, the supported simulator is Gazebo with a custom Python library to interact THEMIS in the simulation environment. The corresponding URDF allowing users to deploy the THEMIS model quickly in a simulator of their choice.

3.8.1 THEMIS Gym

THEMIS Gym is the default simulation environment for THEMIS. It utilizes a custom library to interact THEMIS in **Gazebo** using Python. Please refer to `Main` `PC/THEMIS/THEMIS/Simulation/README.md` for detailed instructions on how to use it. Users can interact with the robot using the same APIs previously mentioned. Note that perception data are not currently available in THEMIS Gym.

4. Troubleshooting

4.1 Actuator Errors

If THEMIS fails to initialize properly, inspect each actuator thread to locate the problem:

Terminal

```
1  screen -r -p bear_right_leg
2  screen -r -p bear_left_leg
3  screen -r -p bear_right_arm
4  screen -r -p bear_left_arm
5  screen -r -p bear_head
6  screen -r -p dxl_right_hand
7  screen -r -p dxl_left_hand
```

4.1.1 BEAR Actuator Errors

Error Name	Solution
Communication	Check the signal cable for continuity.
Overheat	Wait for the motor to cool down.
Absolute Position	In a terminal on the Main Computer, run cd THEMIS/THEMIS followed by python3 -m

	Util.bear_set_posi, then follow the prompts to repair the faulty motor. If the repair fails, rotate that motor to a different angle and try again. Note that recalibration may be required after a successful repair.
E-STOP	Check the E-STOP button.
Joint Limit	Rotate the motor back within its limits.
Hardware Fault	Power-cycle the robot.
Initialization	Power-cycle the robot.
Offline	Check the signal and power cables for continuity.

4.1.2 DYNAMIXEL Actuator Errors

Error Name	Solution
Input Voltage	Check the supply voltage.
Overheat	Wait for the motor to cool down.
Electrical Shock	Check the power cable for continuity.
Overload	Restart the program. If the error recurs, stop operation and inspect the motor load.
Offline	Check the signal and power cables for continuity.

5. Danger Zone



All operations in this section can be **DESTRUCTIVE!**

- Make sure you download all your code, data and any other assets from the robot to your local storage before proceeding.
- Operations in this section may cause the robot or its components to malfunction or become completely disabled. We highly recommend consulting our technical team before proceeding and exercise extreme caution during operation.

5.1 Reflashing the Computers

5.1.1 Main Computer

Flash the main computer with Ubuntu 22.04.5 "Jammy Jellyfish". You can connect the peripherals for the main computer using the ports on the back of THEMIS.

⚠️ TODO: files, settings and dependencies.

Python 3.10.12, pip, NumPy, pyserial, termcolor, numba, posix ipc, lcm, etc.

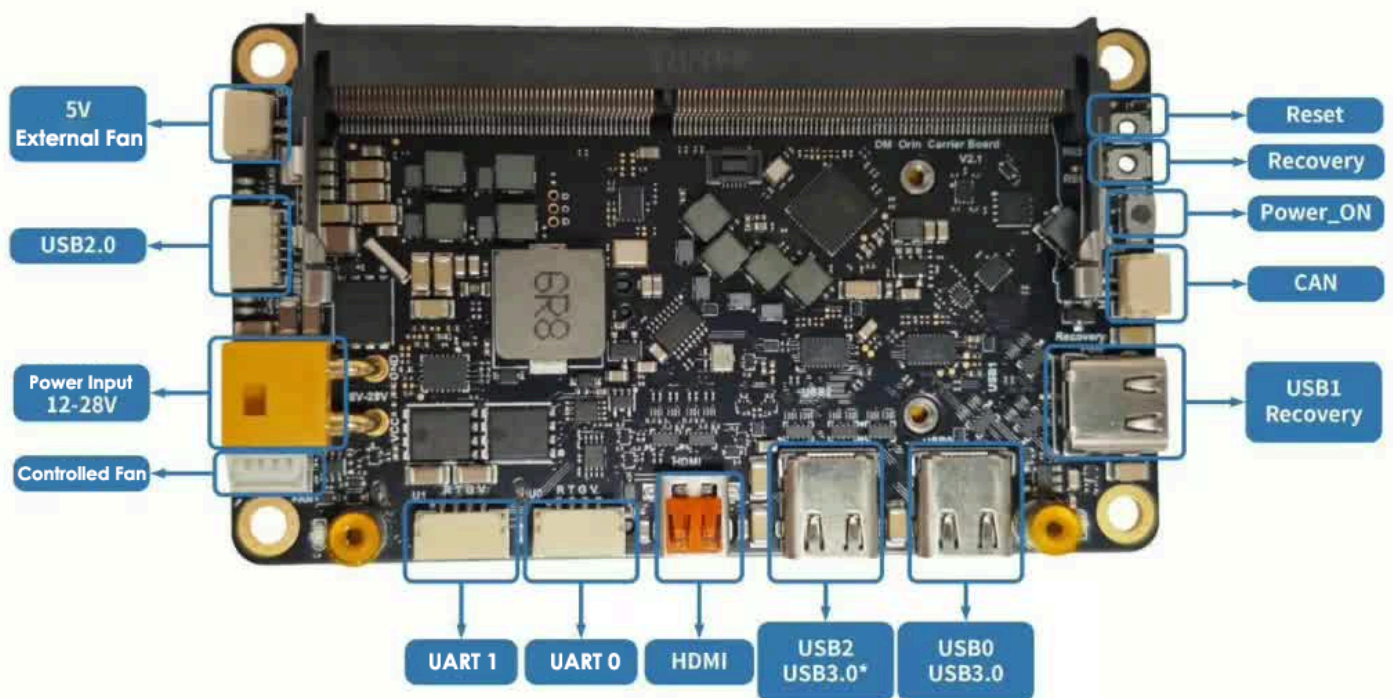
5.1.2 Head Vision Computer

Using [NVIDIA SDK Manager](#) , both the **system image** and [JetPack 6.2](#) will be flashed onto the device storage.

After flashing is complete, the user must manually install the following:

- **ZED SDK**
- **ROS 2**
- The `.deb` packages provided with our factory release

To enter flash mode, press and hold the recovery button --> connect the power supply--> connect USB1 to the host computer via a USB data cable --> release the recovery button. Now users can complete the flashing via the NVIDIA SDK Manager on the host computer following the [steps](#).



⚠️ CAUTION

Back up your data before flashing Jetson!



TODO: How to access ports for flashing?

5.1.3 Front Vision Computer

The front vision computer has a configuration similar to that of the head vision computer. The only difference is that the ZED X driver should be installed on the front computer for the ZED X Mini camera.

Each ZED Link GMSL2 capture card requires the installation of a driver to operate correctly. The required driver package and configuration depend on the Jetson carrier board, deserializer card, and GMSL2 camera setup. Refer to [ZED Link Driver Installation](#) to configure the driver.

Appendix A: Robot Modeling

Refer to [\[archive\] Robot Modeling](#) for details.